Laboratory Guide 3: Quantum Algorithms

Oracles, Phase Kickback, and the Quantum Advantage

Course: Quantum Computing Foundation

1 Introduction and Objectives

In the previous labs, we learned how to manipulate qubits and create entanglement. Now, we will use these tools to perform actual computation. We will implement **Deutsch's Algorithm**, the simplest algorithm that proves a quantum computer can solve a problem faster than any classical computer.

Prerequisites:

- Lesson QC12: *Algorithms - Quantum Parallelism*
- Lesson QC13: *Deutsch's Algorithm*
- Lab 2: *Multi-Qubit Systems*

Learning Objectives:

1. Understand the Query Model and the concept of a Quantum Oracle.
2. Visualize the "Phase Kickback" mechanism using the Composer.
3. Implement Deutsch's Algorithm in Qiskit to determine if a function is Constant or Balanced.
4. Comprehend the practical meaning of "Quantum Advantage".

Important: The Black Box (Oracle)

What is an Oracle?
In computer science, an Oracle is a "Black Box". We can give it an input x and it returns an output $f(x)$, but we **cannot** look inside to see how it calculates the answer. Our goal is to discover properties of $f(x)$ using the minimum possible number of queries (questions) to the Oracle.

2 Part A: The Secret Mechanism (IBM Quantum Composer)

Platform: IBM Quantum Composer

2.1 Exercise 1: Phase Kickback

Context: As explained in Lesson **QC13**, quantum parallelism evaluates $f(0)$ and $f(1)$ simultaneously. But to extract the answer, we need **Phase Kickback**. This phenomenon occurs when a target qubit is in the state $|-\rangle$. Applying a CNOT gate in this condition transfers a phase change *back* to the control qubit, rather than flipping the target!

Student Activity

Action:

1. Add an `H` gate to q_0 (Control). It is now in state $|+\rangle$.
2. Add an `X` gate, then an `H` gate to q_1 (Target). It is now in state $|-\rangle$.
3. Apply a `CNOT` gate (Control q_0, Target q_1).
4. **Look at the Q-Sphere** before and after the CNOT.

Expected Effect: Normally, a CNOT flips the target. But here, the target q_1 remains $|-\rangle$. Instead, the phase of the control qubit q_0 is flipped from $|+\rangle$ to $|-\rangle$! The "action" kicked back to the control.

IBM Composer View (Q-Sphere)

Q-Sphere Visualization
Before the CNOT, the dots on the Q-Sphere for the control qubit are Blue (Positive phase). After the CNOT, the dot corresponding to the $|1\rangle$ component of the control qubit turns **Red** (Negative phase). The target qubit's probabilities did not change at all.

3 Part B: Coding Deutsch's Algorithm (Qiskit)

3.1 Exercise 2: Finding the Function Type

Context: We are given a function $f(x)$ that takes 1 bit (0 or 1) and returns 1 bit (0 or 1). There are two types of functions:

- **Constant:** Returns the same value for all inputs (e.g., always 0, or always 1).
- **Balanced:** Returns 0 for half the inputs and 1 for the other half.

Classically, we must check both $f(0)$ and $f(1)$ (2 queries). Quantumly, we can do it in 1 query.

Coding Task: Deutsch's Algorithm

```python
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram

# 1. Setup: 2 Qubits (q0 is input, q1 is ancilla/target)
qc = QuantumCircuit(2, 1)

# 2. Initialization
qc.h(0)            # Put input in superposition |+>
qc.x(1)            # Put ancilla in |1>
qc.h(1)            # Put ancilla in |-> for phase kickback

qc.barrier()       # Visual separator

# 3. THE ORACLE (Example: A Balanced Function)
# We implement f(x) = x. This is a CNOT gate.
qc.cx(0, 1)

qc.barrier()

# 4. Interference and Measurement
qc.h(0)            # Mix the phases
qc.measure(0, 0)   # Measure the input qubit

# 5. Execute
job = AerSimulator().run(qc, shots=1000)
counts = job.result().get_counts()
print(f"Result: {counts}")
```

Analysis: If the result is **0**, the function is Constant. If the result is **1**, the function is Balanced. Since we used a CNOT (which acts differently for 0 and 1), the function is Balanced, and your output should be 100% '1'.

Deep Dive: Beyond Deutsch (QC14)

Beyond Deutsch: Why does this matter? (Lesson QC14)
You might think: *"Who cares if a 1-bit function is constant or balanced?"* Deutsch's algorithm has no direct practical application. However, it proves that quantum interference can solve a problem in $\mathcal{O}(1)$ steps instead of $\mathcal{O}(N)$.

- **Deutsch-Jozsa:** Expands this to N bits. The classical computer needs $2^{N-1} + 1$ queries. The quantum computer still needs exactly **1 query**. This is an exponential speedup!
- **Grover's Algorithm:** Uses similar Oracle/Phase kickback logic to search unsorted databases quadratically faster.
- **Shor's Algorithm:** Uses these principles to find prime factors, fundamentally threatening modern RSA cryptography.

4 Part C: Assessment (Quiz)

Q1. The Power of the Oracle
In the Query Model of computation (QC12), what is the primary metric used to evaluate the efficiency of a quantum algorithm?

A) The clock speed of the quantum processor.

B) The number of qubits used.

C) The number of times the algorithm queries the Oracle function $f(x)$.

Q2. Phase Kickback Mechanism
In Deutsch's algorithm, what is the required state of the "ancilla" (target) qubit before querying the Oracle to ensure Phase Kickback works?

A) $|0\rangle$ (Ground state)

B) $|-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$

C) $|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}$

Q3. Reading the Output
You run Deutsch's algorithm on a mysterious Oracle and measure the input qubit at the very end. The result is 100% certainty of reading **"0"**. What did the algorithm just prove about the hidden function?

A) The function is Balanced (e.g., $f(x) = x$).

B) The function is Constant (e.g., $f(x) = 1$ for all x).

C) The Oracle contains an error.

Solutions: Q1: C (As stated in QC12); Q2: B (The minus sign is required to kick back the phase); Q3: B.